Module Scripts

Module Scripts allows you to add more complex and specific handling of values, help parsing data from network or serial streams that are following a specific API.

It's also the way to create Custom Modules, where you can define your own values and parameters, and have scripts run the main logic.

Module common functions

All scripts running inside a module will have a common set of functions that can be added to the script. Those functions are not needed to run the script.

Method	Description	Example
function moduleParameterChanged(*param***)**	This function will be called each time a parameter of this module has changed, meaning a parameter or trigger inside the "Parameters" panel of this module.	`function moduleParameterChanged(param){ script.log("Param changed : "+param.name); }`
function moduleValueChanged(*param***)**	This function will be called each time a value of this module has changed, meaning a parameter or trigger inside the "Values" panel of this module.	`function moduleValueChanged(value) { if(value.isParameter()) { script.log("Module value changed : "+value.name+" > "+value.get()); }else { script.log("Module value triggered : "+value.name); } }`

Module-specific callback functions

Some modules have specific function callbacks that are useful if you want to customize the parsing of received data from this module.

▼ OSC

Method	Description	Example
oscEvent(*address, args, originIp***)**	This function will be called each time an OSC message is received. ***address*** is the address of the OSC Message ***args*** is an array containing all the arguments of the OSC Message ***originIp*** is the ip address of the sender	`function oscEvent(address, args) { script.log("OSC Message received "+address+", "+args.length+" arguments"); }`

▼ MIDI

Method	Description	Example
noteOnEvent (*channel, pitch, velocity*)	This function will be called each time a noteOn event is received. The arguments are respectively the channel, pitch and velocity of this event.	`function noteOnEvent(channel, pitch, velocity) { script.log("Note on received "+channel+", "+pitch+", "+velocity); }`
noteOffEvent (*channel, pitch, velocity*)	This function will be called each time a noteOff event is received. The arguments are respectively the channel, pitch and velocity of this event.	`function noteOffEvent(channel, pitch, velocity) { script.log("Note off received "+channel+", "+pitch+", "+velocity); }`
ccEvent (*channel, number, value*)	This function will be called each time a noteOff event is received. The arguments are respectively the channel, number and value of this event.	`function ccEvent(channel, number, value) { script.log("ControlChange received "+channel+", "+number+", "+value); }`
sysExEvent(*data*)	This function will be called each time a sysEx event is received. The argument is an array of bytes containing the sysEx data.	Explain `function sysExEvent(data) { script.log("Sysex Message received, "+data.length+" bytes :"); for(var i=0; i < data.length; i++) { script.log(" > "+data[i]); } }`
channelPressureEvent (*channel, value*)	This function will be called each time a channelPressure event is received. The arguments are respectively the channel and the value of this event.	`function channelPressureEvent(channel, value) { script.log("Channel Pressure received "+channel+", "+value); }`
afterTouchEvent (*channel, note, value*)	This function will be called each time an afterTouch event is received. The arguments are respectively the channel, the note and the value of this event.	`function afterTouchEvent(channel, note, value) { script.log("After Touch received "+channel+", "+note+", "+value); }`

▼ MQTT

Method	Description	Example
dataEvent(*data, topic*)	This function will be called each time a MQTT message is received. ***data*** is the payload of the MQTT message ***topic*** is the topic of the MQTT message	`function dataEvent(data, topic) { script.log("MQTT Message received " + topic + " : " + data); }`

▼ DMX

Method	Description	Example
dmxEvent(*values*)	This is called when a group of DMX channel values is received. *values* is an array of values, always starting from channel 1	`function dmxEvent(values) { script.log("Received dmx : "+values.length+" values"); }`

▼ Serial / UDP / TCP

Method	Description	Example
dataReceived(*data*)	This function will be called each time data has been received. If the Module's protocol is set to ***Lines,*** then the **data** argument will be a string containing the line, without the ending \n. Otherwise, the data will be an array of bytes containing the received data.	`function dataReceived(data) { script.log("Received data : "+data); }`

Pierre Lemmel 2025/05/27 （已编辑）
I’m not sure if this function has a client parameter that was not indicated before

Benjamin Kuperberg 2025/05/29
this is actually very inconsistent right now :
client and server don’t have the ... 更多

▼ WebSocket

Method	Description	Exemple
wsDataReceived(*data***)**	This function will get called each time a binary data packet is received	`fonction wsDataReceived(data) {script.log("Data received : "+data.length);}`
wsMessageReceived(*client*, *message***)**	This function will be called each time a string message is received	`fonction wsMessageReceived(client, message) {script.log("Message received: "+message);}`

▼ HTTP

Method	Description	Exemple
dataEvent(*data, requestURL***)**This function will be called each time data has been received.	This function will be called each time the module has got a response from a request. ***data*** is the content of the response ***requestURL*** is the url of the original request.	`function dataEvent(data, requestURL) { script.log("Data received, request URL :"+requestURL+"\nContent :\n" +data); }`

▼ System

Method	Description	Example
processDataReceived(*data, originCommand***)**	This function will be called after a call to ***launchProcess()*** with blocking set to *false*. ***data*** contains the output of the process as a string ***originCommand*** contains the command that was called for this output	`function processDataReceived(data, command) { script.log("Data received",data,command); }`

Module-specific methods (the local object)

Some modules have specific methods that are useful if you want to have specific behaviors or send complex or automated data. Those methods are part of the **local** Javascript object, depending on the module it's running in.

▼ OSC

Method	Description	Example
send(*address, arg1, arg2, arg3, ...***)**	This sends an OSC message to all the enabled outputs of this module. ***address*** is the address of the message	`local.send("myAddress", 1, .5f, "cool");`
sendTo(*ip, port, address, arg1, arg2, ...***)**	Same as the **send** method, but will send to a specific ***ip*** and ***port***, ignoring the module's output.	`local.sendTo("192.168.0.30", 9000, "myAddress", 1, .5f);`
match(*address, pattern***)**	Check if the specified ***address*** matches the ***pattern***, according to the OSC specification. Wildcards, etc are supported.	`if (local.match(address, "/testPattern")) ...`

register(*pattern, callbackFunc***)**	Register a script function to call when a message matching ***pattern*** is received. ***callbackFunc*** is the name of the callback function. **[Note]** if the module contains multiple scripts, the callback is registered for all the scripts.	`function init() { local.register("/testPattern", "testPatternCallback"); }`
		
		`function testPatternCallback(address, args) { script.log("Received message "+address); }`

▼ MIDI

Method	Description	Example
sendNoteOn(*channel, pitch, velocity***)**	This will send a Note On event on the module's output MIDI device.	`local.sendNoteOn (1, 12, 127);`
sendNoteOff(*channel, pitch***)**	This will send a Note Off event on the module's output MIDI device.	`local.sendNoteOff (1, 12);`
sendCC(*channel, pitch, velocity***)**	This will send a Control Change event on the module's output MIDI device.	`local.sendCC (3, 20, 65);`
sendSysex(*byte1, byte2, byte3, ...***)**	This will send a Sysex message on the module's output MIDI device. You can add as many arguments you want in the Sysex message.	`local.sendSysex (10, 15,20,20,0);`
sendPitchWheel(*channel, value***)**	This will send a Pitch Wheel event on the module's output MIDI device.	`local.sendPitchWheel (3, 2000);`
sendChannelPressure(*channel, value***)**	This will send a Channel Pressure event on the module's output MIDI device.	`local.sendChannelPressure (1, 67);`
sendAfterTouch(*channel, note, value***)**	This will send an After Touch event on the module's output MIDI device.	`local.sendAfterTouch (3, 20, 65);`
sendProgramChange(*channel, program*)	This will send a Program Change event on the module's output MIDI device.	`local.sendProgramChange((5, 99)`

▼ DMX

Method	Description	Example
send(*startChannel, value1, value2, ..., valueN***)**	Sends DMX values starting at the **startChannel** to universe defined in the module's parameters. You can add as many values as you want, and you can even mix array of values with single values. Values are 0 to 255.	`local.send(32, 255);`
sendUniverse(*net, subnet, universe, startChannel, value1, value2, ..., valueN*)	Send DMX values to choosen universe, overrring the module's parameters. Values are the same than with `send()`	`local.sendUniverse(0, 4, 1, 32, 255);` Here sending the value `255` to channel `32` of universe `1` on net `0` / subnet `4`.

▼ Serial / UDP / TCP / WebSocket

Method	Description	Example
send(*message***)**	This will send the string passed in as ASCII characters.	`local.send("This is my message");`
sendBytes(*byte1, byte2, [bytes], ...***)**	This will send all the bytes passed in as they are. The bytes can be packed into arrays or just laid out one by one, or a mix of arrays and bytes.	`local.sendBytes(30,210, [255,0,0], 5);`
sendTo(*ip, port, message***)**	**[UDP Only]** Same as the **send** method, but will send to a specific ***ip*** and ***port***, ignoring the module's output.	`local.sendTo("192.168.1.255", 8888, "This is my message");`
sendBytesTo(*ip, port, byte1, byte2, [bytes], ...***)**	**[UDP Only]** Same as the **sendBytes** method, but will send to a specific ***ip*** and ***port***, ignoring the module's output.	`local.sendBytesTo("192.168.1.255", 8888, 30,210, [255,0,0], 5);`

▼ HTTP

There are 2 ways of using following methods.

In all those methods, the ***address*** field is appended to the modules "Base Address" parameter. The examples are considering that default base address is https://httpbin.org/

Inline arguments

You can either pass all the arguments inline, in which case some things are not possible (putting extra headers and payload in other requests than GET for example).

Method	Description	Example
sendGET(*url, [dataType], [extraHeaders], [payload]***)**	This will send an HTTP GET request. You can add parameters at the end of the ***url*** argument, just like any GET URL. You can add optional parameters after the address (only works with GET) : • dataType (***json*** or ***raw***) : defines what type of data to expect for the result • extraHeaders : add extra headers to the query • payload : • add payload to the query	`local.sendGET( "anything?myValue1=1&myValue2=super", "json", "Connection: keep-alive", "some payload here");`
sendPOST(*url, param1, value1, param2, value2, ...***)**	This will send an HTTP POST request. After specifying the ***url***, you can add pair of values that are respectively the ***parameter name*** and its ***value***.	`local.sendPOST( "anything", "myValue1", 1, "myValue2", 2);`
sendPUT(*url, param1, value1, param2, value2, ...***)**	This will send an HTTP PUT request. After specifying the ***url***, you can add pair of values that are respectively the ***parameter name*** and its ***value***.	`local.sendPUT( "anything", "myValue1", 1, "myValue2", 2);`
sendPATCH(*url, param1, value1, param2, value2, ...***)**	This will send an HTTP PATCH request. After specifying the ***url***, you can add pair of values that are respectively the ***parameter name*** and its ***value***.	`local.sendPATCH( "anything", "myValue1", 1, "myValue2", 2);`
sendDELETE(*url, param1, value1, param2, value2, ...***)**	This will send an HTTP DELETE request. After specifying the ***url***, you can add pair of values that are respectively the ***parameter name*** and its ***value***.	`local.sendDELETE( "anything", "myValue1", 1, "myValue2", 2)`

Object arguments

You can pass an object containing the parameters to send the request. All properties of the "params" object are optional. When adding all, this looks like this :

Copy